HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO NỘI DUNG

Nội dung 2: Phân tích và thiết kế hệ thống

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Chủ trì đề tài: Trần Đức Định (B23DCCC038)
Thành viên: Hoàng Nguyễn Trà My (B23DCCC116)
Giảng viên hướng dẫn: TS. Lê Văn Hoàng

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG.....1
1.1. Nhiệm vụ theo đề cương.....1
1.2. Phương pháp thực hiện.....1
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....2
2.1. Khảo sát và xác định yêu cầu.....2
2.1.1. Đối tượng người dùng.....2
2.1.2. Quy trình họp mẫu.....2
2.2. Yêu cầu chức năng.....3
2.3. Yêu cầu phi chức năng.....4
2.4. Mô hình ca sử dụng.....5
2.5. Kiến trúc tổng thể.....7
2.5.1. Các thành phần.....7
2.5.2. Các nguyên tắc kiến trúc.....8
2.6. Luồng xử lý từ âm thanh đến công việc.....9
2.6.1. Luồng tổng quát.....9
2.6.2. Hợp đồng sự kiện.....11
2.6.3. Trạng thái xử lý.....12
2.7. Thiết kế cơ sở dữ liệu.....13
2.7.1. Tổ chức schema và tài khoản.....13
2.7.2. Các bảng chính.....13
2.7.3. Lưu phiên bản transcript và biên bản.....15
2.7.4. Nhãn người nói và bí danh.....15
2.7.5. Trích dẫn nguồn của công việc.....15
2.8. Thiết kế phân quyền.....15
2.8.1. Vai trò theo phạm vi.....15
2.8.2. Trình tự kiểm tra quyền.....17
2.9. Thiết kế bộ phân tích AI.....17
2.9.1. Quy tắc chống bịa trong prompt.....17
2.9.2. Cấu trúc đầu ra.....18
2.9.3. Con người trong vòng lặp.....18
2.9.4. Yêu cầu thiết kế cho hỏi đáp tri thức.....18
2.10. Thiết kế giao diện.....19
2.11. Thiết kế module đồng bộ công cụ ngoài và cảnh báo.....20
2.11.1. Hiện trạng và dữ liệu.....20
2.11.2. Nguyên tắc chống tạo trùng và cảnh báo.....20
2.12. Kết luận chương.....21
CHƯƠNG 3. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ.....22
3.1. Đối chiếu với kết quả cần đạt.....22
TÀI LIỆU THAM KHẢO.....23

DANH MỤC BẢNG BIỂU

Bảng 2.1. Các tác nhân của hệ thống.....2
Bảng 2.2. Yêu cầu chức năng và trạng thái hiện thực.....3
Bảng 2.3. Yêu cầu phi chức năng và giải pháp thiết kế.....4
Bảng 2.4. Đặc tả ca sử dụng "Mời bot vào cuộc họp".....6
Bảng 2.5. Đặc tả ca sử dụng "Duyệt đề xuất công việc".....6
Bảng 2.6. Đặc tả ca sử dụng "Gán tên người nói".....6
Bảng 2.7. Các thành phần của hệ thống.....7
Bảng 2.8. Hợp đồng sự kiện giữa các dịch vụ.....11
Bảng 2.9. Trạng thái của job, cuộc họp và phiên bot.....12
Bảng 2.10. Các bảng chính của schema core.....14
Bảng 2.11. Một số quyền tiêu biểu và vai trò được phép.....16
Bảng 2.12. Cấu trúc JSON đầu ra của bộ phân tích.....18
Bảng 2.13. Các màn hình chính của giao diện web.....19
Bảng 2.14. Các bảng của schema integration.....20
Bảng 3.1. Đối chiếu kết quả Nội dung 2 với đề cương.....22

DANH MỤC HÌNH VẼ

Hình 2.1. Sơ đồ ca sử dụng tổng quát của hệ thống MeetingMind.....5
Hình 2.2. Kiến trúc tổng thể hệ thống MeetingMind.....7
Hình 2.3. Luồng xử lý từ âm thanh đến công việc.....10
Hình 2.4. Sơ đồ tuần tự sự kiện giữa core-service và ai-worker.....11
Hình 2.5. Sơ đồ quan hệ các bảng chính của schema core.....14
Hình 2.6. Phân quyền theo vai trò kết hợp phạm vi tài nguyên.....16

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG

1.1. Nhiệm vụ theo đề cương

Nội dung 2 được thực hiện trong tháng 06/2026, tiếp nối kết quả tổng quan của Nội dung 1. Theo đề cương, nhiệm vụ của bước này là đặc tả yêu cầu hệ thống, thiết kế kiến trúc microservices xử lý âm thanh, thiết kế schema cơ sở dữ liệu và luồng xử lý prompt. Kết quả cần đạt gồm tài liệu đặc tả yêu cầu (Software Requirements Specification, SRS), sơ đồ kiến trúc hệ thống, thiết kế cơ sở dữ liệu và luồng dữ liệu từ âm thanh (Audio) đến công việc (Task).

Bài toán thiết kế của MeetingMind có ba điểm khó. Thứ nhất, một cuộc họp có thể đi vào hệ thống bằng nhiều đường khác nhau (tải file, bot, extension) nhưng phần xử lý phía sau phải dùng chung. Thứ hai, đầu ra của AI không đáng tin tuyệt đối nên không được ghi thẳng thành dữ liệu chính thức, mà phải qua bước con người duyệt. Thứ ba, transcript và biên bản có thể bị sửa nhiều lần, trong khi công việc đã giao phải luôn truy được về đúng câu nói gốc tại thời điểm duyệt.

1.2. Phương pháp thực hiện

Nhóm phân tích yêu cầu từ quy trình họp thực tế của các nhóm phát triển phần mềm (trước, trong và sau cuộc họp), xác định tác nhân và ca sử dụng, sau đó thiết kế theo hướng sự kiện: mỗi bước xử lý dài (phiên âm, phân tích) là một job bất đồng bộ, giao tiếp qua hàng đợi thông điệp. Cơ sở dữ liệu được thiết kế theo nguyên tắc mỗi bảng có đúng một dịch vụ sở hữu và mọi dữ liệu do AI sinh ra đều được lưu theo phiên bản. Các tài liệu thiết kế gốc (tổng quan dự án, kiến trúc, thiết kế cơ sở dữ liệu, hợp đồng sự kiện) được lưu cùng mã nguồn để thiết kế và cài đặt luôn đi cùng nhau.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích yêu cầu và các quyết định thiết kế của MeetingMind: từ khảo sát người dùng và quy trình họp, tới kiến trúc, luồng sự kiện, cơ sở dữ liệu, phân quyền, bộ phân tích AI, giao diện và module tích hợp. Bộ tài liệu thiết kế gốc được soạn cho toàn bộ phạm vi sản phẩm, rộng hơn khối lượng làm được trong sáu tháng, nên ở các bảng yêu cầu và thành phần, chúng tôi ghi trạng thái đã đối chiếu với mã nguồn theo ba mức: *Đã có* (chạy trong luồng chính), *Một phần* (có phần lõi, chưa trọn chức năng) và *Thiết kế* (mới có đặc tả, bảng dữ liệu hoặc giao diện mẫu).

2.1. Khảo sát và xác định yêu cầu

2.1.1. Đối tượng người dùng

Yêu cầu được thu thập từ hai nguồn. Thứ nhất là khảo sát các sản phẩm dùng bot vào phòng họp như Otter.ai [17], Fireflies.ai [6] và sản phẩm đọc phụ đề như Tactiq [23]; chúng ghi nhận và tóm tắt tốt nhưng dừng ở danh sách việc chưa qua kiểm duyệt, ít chú trọng việc ai được xem gì và chưa tối ưu cho tiếng Việt. Thứ hai là phân tích quy trình họp của các nhóm phát triển phần mềm nhỏ, được hệ thống hoá thành tài liệu tổng quan dự án gồm mười lăm mục. Kết quả là bốn nhóm người dùng và hai tác nhân hệ thống ở Bảng 2.1.

Bảng 2.1. *Các tác nhân của hệ thống*

Tác nhân	Mục tiêu chính
Owner/Admin workspace	Quản lý thành viên, dự án, cấu hình, kết nối tích hợp
PM/Trưởng nhóm	Tổ chức họp, gán tên người nói, duyệt biên bản và đề xuất, phân công
Thành viên	Xem cuộc họp được chia sẻ, ghi chú, cập nhật việc được giao
Khách mời	Chỉ xem tài liệu được chia sẻ riêng; có thể không có tài khoản
Bot ghi âm	Vào phòng Google Meet, ghi âm, tải bản ghi lên theo lệnh của Core
Dịch vụ AI	Phiên âm, tách người nói, đề xuất biên bản và công việc

Khảo sát cũng cho thấy cần tách *quyền trên hệ thống*, *vai trò chuyên môn* (Backend, QA…) và *vai trò trong cuộc họp hoặc đầu việc* (chủ trì, người thực hiện). Một người có thể là thành viên thường của workspace nhưng là PM của dự án A; AI được gợi ý người thực hiện nhưng không bao giờ được thay đổi quyền.

2.1.2. Quy trình họp mẫu

Trước họp, người tổ chức tạo cuộc họp trong một dự án hoặc độc lập, nhập agenda, người tham dự và chọn cách thu nhận: mời bot vào Google Meet, bật tiện ích

đọc phụ đề, ghi âm tại phòng họp, hoặc tải bản ghi có sẵn. *Trong họp*, hệ thống thu nhận nội dung; người dự ghi chú cá nhân hoặc chia sẻ. *Sau họp*, hệ thống phiên âm, tách lượt nói, sinh biên bản nháp và đề xuất công việc; người chủ trì gắn tên cho các nhãn "Người nói 1, 2…", sửa và duyệt; chỉ đề xuất đã duyệt mới thành công việc chính thức.

Tình huống chuẩn cho AI: Minh nói *"Nam phụ trách API đăng nhập, thứ Sáu gửi bản đầu tiên để Lan kiểm thử."* Kết quả mong đợi là công việc giao cho Nam, giữ nguyên "thứ Sáu" để người duyệt chuẩn hoá, kèm câu nguồn; hạn kiểm thử của Lan không được nêu nên để trống. Từ đó rút ra các quy tắc nghiệp vụ: phân biệt người nói với người được nhắc tới, đề xuất với cam kết; có đính chính thì lấy thông tin sau cùng; thiếu thông tin thì để trống; mọi công việc có nguồn; chỉ công việc đã duyệt mới được giao.

2.2. Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo vòng đời cuộc họp ở Bảng 2.2. Thứ tự ưu tiên theo phân kỳ đã đặt: phần lõi từ thu nhận tới bảng công việc làm trước; tích hợp ngoài, cảnh báo nâng cao và hỏi đáp làm sau.

Bảng 2.2. *Yêu cầu chức năng và trạng thái hiện thực*

Nhóm	Chức năng	Trạng thái
Tài khoản, workspace	Đăng ký kèm tạo workspace, đăng nhập, đổi mật khẩu; mời thành viên với vai trò	Đã có
	Tạo dự án; quản lý thành viên dự án	Một phần
	Mẫu biên bản, lưu trữ, nhắc việc, nhật ký quản trị	Thiết kế
Cuộc họp	Tạo cuộc họp kèm agenda, người dự, nguồn thu nhận; danh sách theo trạng thái	Đã có
	Ghi chú cá nhân/chia sẻ; chủ ghi chú cho phép đưa vào biên bản	Đã có
	Kết nối lịch, bot tự vào họp theo lịch	Thiết kế
Thu nhận nội dung	Tải tệp ghi âm, chống trùng theo SHA-256, URL ký để tải lại	Đã có
	Bot ghi âm Google Meet; tiện ích Chrome đọc phụ đề	Đã có
	Ghi âm trên trình duyệt tại phòng họp, ghi khi mất mạng	Thiết kế
Transcript, người nói	Phiên âm tiếng Việt xen tiếng Anh, tách người nói, đánh dấu nói chồng	Đã có
	Transcript theo phiên bản; gán nhãn người nói, giữ "chưa xác định"	Đã có
	Sửa nội dung đoạn, gộp/tách nhãn, phát lại audio đồng bộ	Một phần
	Nhận diện qua mẫu giọng đã đăng ký	Thiết kế

Nhóm	Chức năng	Trạng thái
Biên bản, duyệt	Biên bản nháp do AI sinh; nháp → chờ duyệt → đã duyệt	Đã có
	Hàng đợi duyệt: sửa, duyệt, từ chối, giữ bản gốc của AI	Đã có
	Xuất biên bản ra tệp hoặc Notion/Google Docs	Thiết kế
Công việc	Duyệt đề xuất sinh công việc; Kanban 4 trạng thái; bình luận	Đã có
	Liên kết công việc với câu nguồn	Một phần
	Lịch sử thay đổi, phụ thuộc giữa công việc	Thiết kế
Tích hợp ngoài	Đồng bộ một chiều công việc đã duyệt sang Trello, thông báo Telegram (đã có); đồng bộ sang Jira (chưa có)	Một phần
Cảnh báo, hỏi đáp	Cảnh báo theo quy tắc: quá hạn, bị chặn, thiếu người phụ trách	Một phần
	Tìm kiếm trong transcript một cuộc họp	Đã có
	Hỏi đáp có trích nguồn (RAG), lọc quyền trước truy xuất; tải tài liệu cho cuộc họp	Đã có

Các mục "Một phần" có lý do cụ thể: sửa đoạn transcript đã có API nhưng chưa có giao diện; liên kết nguồn của công việc đi qua đề xuất gốc, bảng trích dẫn riêng chưa được ghi tự động; cảnh báo được tính ở phía giao diện, chưa có bộ quét phía máy chủ; tích hợp ngoài mới có Trello và Telegram, cấu hình qua biến môi trường, chưa có Jira. Nhóm chủ động làm trọn luồng lõi trước, vì nếu luồng này sai thì đồng bộ ra ngoài chỉ nhân bản sai sót.

2.3. Yêu cầu phi chức năng

Với hệ thống xử lý hội thoại nội bộ bằng mô hình AI có thể sai, yêu cầu phi chức năng quan trọng ngang chức năng; Bảng 2.3 tổng hợp các giải pháp.

Bảng 2.3. *Yêu cầu phi chức năng và giải pháp thiết kế*

Nhóm	Giải pháp	Trạng thái
Bảo mật	JWT, mật khẩu băm bcrypt, phân quyền theo phạm vi tập trung ở Core; khoá API ở biến môi trường	Đã có
	Mã hoá dữ liệu hội thoại khi lưu	Chưa triển khai
Toàn vẹn dữ liệu	Transcript, biên bản theo phiên bản; đề xuất giữ bản gốc của AI	Đã có
Tin cậy	Outbox, bảng chống xử lý lặp, khoá idempotency cho job	Đã có
	Tự động thử lại job lỗi, hàng đợi thư chết	Thiết kế
Hiệu năng	Phiên âm, phân tích chạy nền qua hàng đợi; giao diện hiển thị trạng thái từng bước	Đã có
Mở rộng	Worker AI và bot tách container, nhận việc từ hàng đợi bền	Đã có

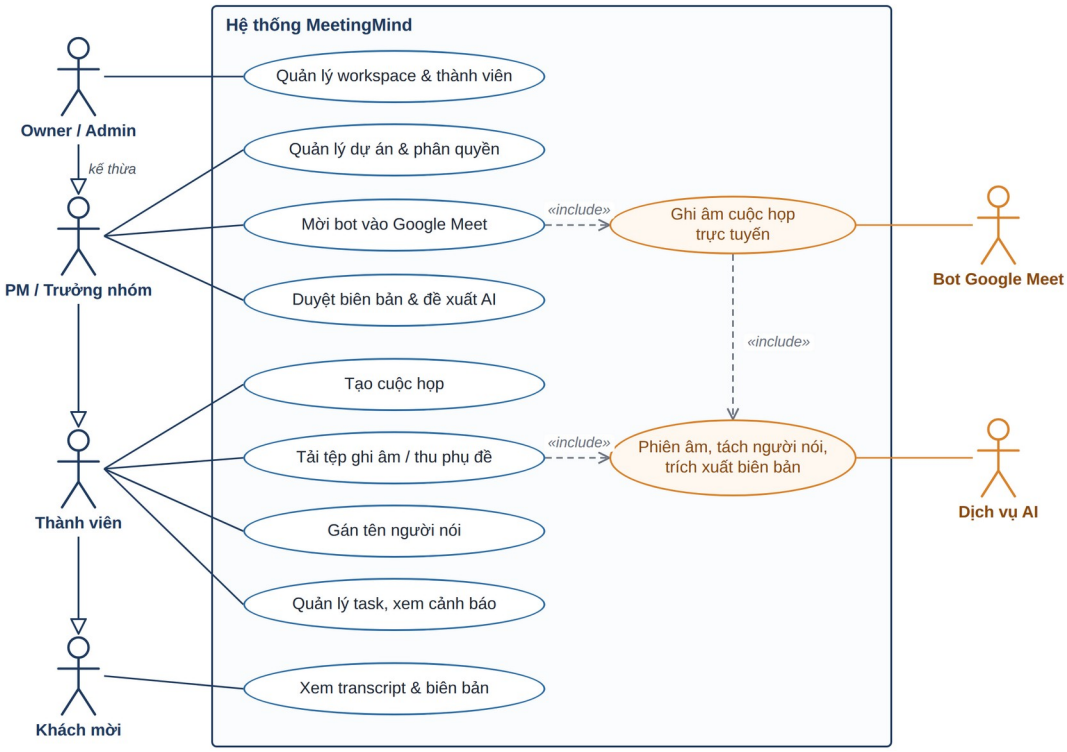
Nhóm	Giải pháp	Trạng thái
Thay thế mô hình AI	Giao diện Transcriber/Analyzer chọn bằng biến môi trường; mô hình chính và dự phòng	Đã có
Quan trắc	correlationId xuyên suốt; Prometheus, Grafana	Một phần

Về bảo mật, chúng tôi đã vá các lỗ hổng của mã khung ban đầu: route quản lý người dùng mở không cần đăng nhập, đổi mật khẩu không kiểm tra mật khẩu cũ, người dùng tự đặt chủ sở hữu workspace. Điểm thiếu lớn nhất là mã hoá khi lưu: transcript và audio hiện chỉ được bảo vệ bằng phân quyền và tài khoản cơ sở dữ liệu riêng.

Về tin cậy, RabbitMQ chỉ bảo đảm giao *ít nhất một lần* [18], nên thiết kế dùng mẫu *transactional outbox* [20] cùng chống lặp phía người nhận và khoá idempotency phía job [13]. Về thay thế mô hình, khi chuyển từ bản giả lập sang faster-whisper [22] và mô hình của FPT AI Factory [7], luồng điều phối ở Core không phải sửa.

2.4. Mô hình ca sử dụng

Hình 2.1 trình bày sơ đồ ca sử dụng tổng quát, trong đó PM tham gia nhiều ca nhất, còn bot và dịch vụ AI chỉ hành động theo lệnh của Core. Bảng 2.4, Bảng 2.5 và Bảng 2.6 đặc tả ba ca tiêu biểu về phối hợp giữa con người, bot và AI.



Hình 2.1. *Sơ đồ ca sử dụng tổng quát của hệ thống MeetingMind*

Bảng 2.4. *Đặc tả ca sử dụng "Mời bot vào cuộc họp"*

Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER hoặc người tạo cuộc họp; bot ghi âm
Tiền điều kiện	Cuộc họp có đường dẫn Google Meet; người dùng có quyền `meeting.recording.control`
Luồng chính	1. Bấm "Mời bot". 2. Core tạo phiên bot `queued`, ghi `bot.join.requested` vào outbox. 3. Bot vào phòng, báo `waiting_admission`; được cho vào thì báo `recording`. 4. Họp kết thúc hoặc bot ở một mình 60 s thì rời phòng, báo `left`. 5. Bot tải bản ghi lên MinIO, phát `recording.completed`. 6. Core đăng ký tệp, khởi động phiên âm
Luồng thay thế	Đã có phiên đang chạy: trả về phiên đó. Dừng trước khi vào phòng: `cancelled`. Bị từ chối hoặc chờ quá 300 s: `failed` kèm lý do

Bảng 2.5. *Đặc tả ca sử dụng "Duyệt đề xuất công việc"*

Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER (người tạo cuộc họp chỉ được sửa)
Tiền điều kiện	Đã có lô duyệt với các đề xuất `pending`
Luồng chính	1. Mở "Task cần duyệt"; đề xuất hiển thị cạnh câu nguồn, gợi ý người thực hiện, cụm hạn gốc. 2. Người duyệt sửa tiêu đề, chọn người thực hiện, ngày hạn, ưu tiên rồi bấm "Duyệt". 3. Core kiểm tra `review.approve`, tạo công việc `todo`, bật cờ cần xác nhận nếu chưa có ngày, chuyển đề xuất sang `approved`
Luồng thay thế	Từ chối kèm lý do: `rejected`. Đề xuất loại quyết định: tạo bản ghi quyết định. Duyệt lặp: ràng buộc duy nhất chặn công việc thứ hai

Bảng 2.6. *Đặc tả ca sử dụng "Gán tên người nói"*

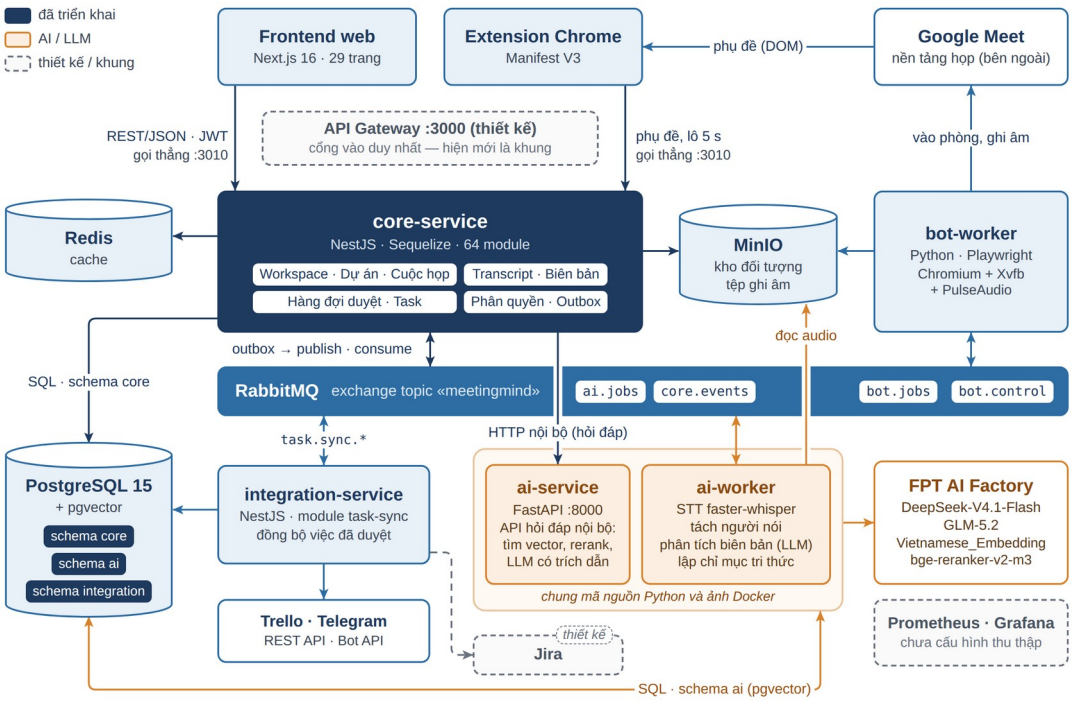
Mục	Nội dung
Tác nhân	PM, Owner/Admin, MANAGER/EDITOR hoặc người tạo cuộc họp
Tiền điều kiện	Có phiên bản transcript hiện hành với nhãn người nói tự động
Luồng chính	1. Mở tab transcript, chọn thành viên cho một nhãn. 2. Core kiểm tra `transcript.update` và tư cách thành viên. 3. Mọi đoạn mang nhãn chuyển sang `confirmed`; ghi `speaker_aliases` kèm người xác nhận. 4. Công việc của cuộc họp được đánh dấu "cần rà soát", không bị đổi người âm thầm
Luồng thay thế	Chọn "Giữ chưa xác định": không đổi. Người được chọn ngoài workspace: báo lỗi

Phân biệt được hai giọng không có nghĩa là biết họ là ai: đường bot và tải tệp chỉ trả nhãn ở trạng thái `unknown`; tên hiển thị trên Google Meet từ đường phụ đề có thể đặt tuỳ ý nên chỉ ở trạng thái `suggested` cho tới khi được xác nhận.

2.5. Kiến trúc tổng thể

2.5.1. Các thành phần

MeetingMind theo kiến trúc nhiều dịch vụ [16], giao tiếp đồng bộ qua HTTP và bất đồng bộ qua RabbitMQ. Hình 2.2 mô tả kiến trúc tổng thể, Bảng 2.7 liệt kê thành phần và trạng thái. Hệ thống chạy bằng Docker Compose với mười hai container; giao diện web chạy riêng.



Hình 2.2. *Kiến trúc tổng thể hệ thống MeetingMind*

Bảng 2.7. *Các thành phần của hệ thống*

Thành phần	Công nghệ	Trách nhiệm	Trạng thái
Giao diện web	Next.js	Cuộc họp, transcript, biên bản, duyệt, công việc	Đã có
Tiện ích Chrome	Manifest V3 [8]	Đọc phụ đề Google Meet	Đã có
api-gateway	NestJS	Cổng vào duy nhất, kiểm tra token	Khung
core-service	NestJS [15], Sequelize	Nghiệp vụ, phân quyền, điều phối	Đã có
ai-worker	Python, faster-whisper, sherpa-onnx	Phiên âm, tách người nói, phân tích LLM, lập chỉ mục tri thức	Đã có
ai-service	FastAPI	API hỏi đáp nội bộ (chỉ Core gọi): tìm vector, xếp hạng lại, LLM trả lời có trích dẫn	Đã có
bot-worker	Python, Playwright [14]	Vào Google Meet, ghi âm, tải lên	Đã có
integration-	NestJS	Đồng bộ công việc sang	Đã chạy (đồng

Thành phần	Công nghệ	Trách nhiệm	Trạng thái
service		Trello, báo Telegram; Jira, lịch (thiết kế)	bộ Trello/Telegram)
PostgreSQL, RabbitMQ, MinIO, Redis	pgvector; exchange topic; tương thích S3	Dữ liệu, hàng đợi, lưu audio, bộ đệm	Đã có
Prometheus, Grafana		Giám sát	Một phần

Cần nói rõ rằng api-gateway mới là khung NestJS chưa có nghiệp vụ; integration-service đã có mô-đun đồng bộ Trello và Telegram nhưng chưa có Jira và kết nối lịch; ai-service và ai-worker dùng chung mã nguồn: ai-worker nhận việc qua RabbitMQ (phiên âm, phân tích, lập chỉ mục tri thức), còn ai-service phục vụ API hỏi đáp đồng bộ, chỉ core-service được gọi qua mạng nội bộ kèm khoá bí mật dùng chung.

2.5.2. Các nguyên tắc kiến trúc

Cổng vào duy nhất. Theo thiết kế, chỉ api-gateway được mở ra ngoài; gateway kiểm tra token và định tuyến nhưng không chứa logic phân quyền, để quyết định "ai được làm gì" chỉ nằm ở Core. Hiện nguyên tắc này mới là mục tiêu: trong môi trường phát triển, giao diện web và tiện ích Chrome gọi thẳng core-service.

Mỗi bảng một dịch vụ sở hữu. Chỉ dịch vụ sở hữu được ghi vào bảng; dịch vụ khác lấy dữ liệu qua API hoặc sự kiện; khoá ngoại chỉ dùng trong một schema. Nguyên tắc này xuất phát từ sự cố thật: khi bốn dịch vụ dùng chung một schema và một tài khoản, dịch vụ AI chiếm mất tên bảng `users` và các dịch vụ NestJS cùng sửa cấu trúc bảng lúc khởi động.

Core điều phối, AI chỉ đề xuất. Bot và AI không tự sửa bảng nghiệp vụ; chúng báo kết quả bằng sự kiện và Core quyết định có ghi thành dữ liệu chính thức hay không. Kết quả phân tích chỉ thành biên bản nháp và đề xuất chờ duyệt, như một cách đối phó với hiện tượng ảo giác của mô hình ngôn ngữ [11]: chấp nhận mô hình có thể sai và đặt con người vào điểm quyết định.

Thông điệp mang tham chiếu, trừ các ngoại lệ có chủ đích. Hợp đồng ban đầu quy định thông điệp chỉ mang định danh, số phiên bản và khoá đối tượng; lệnh phiên âm giữ đúng quy định, ai-worker tự tải tệp từ MinIO. Ở bước phân tích, quy định này mâu thuẫn với nguyên tắc sở hữu: transcript nằm trong schema `core` mà ai-worker không được đọc; cấp quyền đọc chéo thì phá ranh giới, gọi ngược API của Core thì tạo phụ thuộc đồng bộ. Chúng tôi quyết định cho `analysis.requested` mang kèm các đoạn transcript, vì văn bản nhỏ hơn nhiều so với audio, vì thông điệp tự đủ nên phân tích luôn chạy trên đúng phiên bản chụp lúc phát lệnh, và vì Core thay được nhãn tự

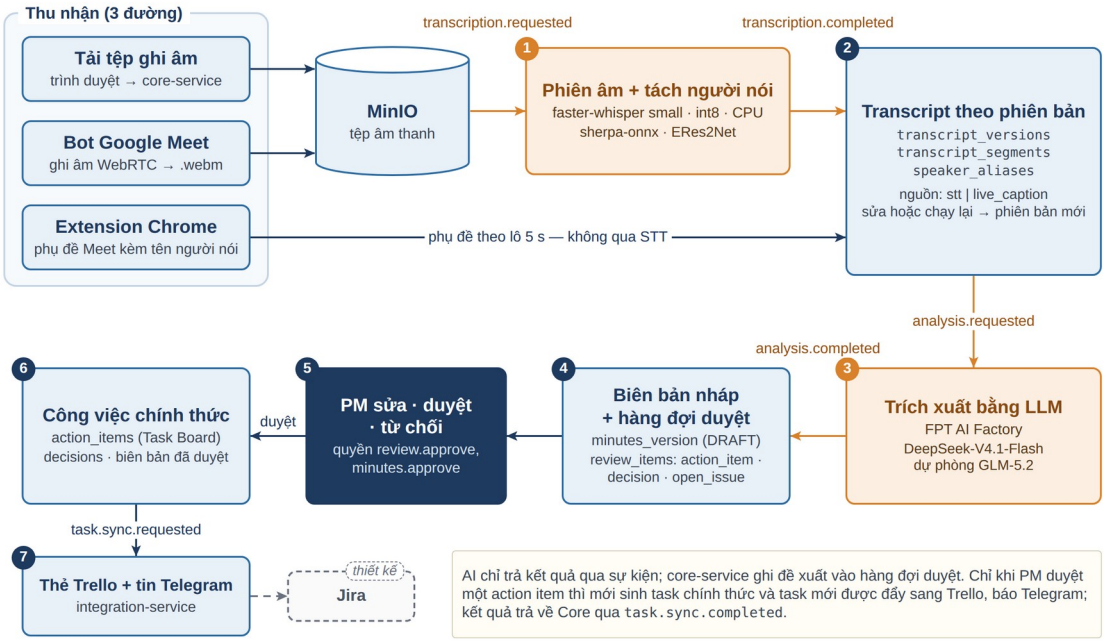
động bằng tên đã gán. Lệnh đồng bộ `task.sync.requested` là ngoại lệ thứ hai với cùng lý do: integration-service không được đọc schema `core`, nên thông điệp mang ảnh chụp công việc tại thời điểm duyệt. Lệnh lập chỉ mục tri thức `knowledge.index.requested` là ngoại lệ thứ ba: nó mang sẵn nội dung transcript, biên bản, quyết định hay công việc cần lập chỉ mục; riêng tài liệu tải lên vẫn theo quy định gốc, thông điệp chỉ mang khoá object để ai-worker tự tải từ MinIO. Các ngoại lệ này được ghi rõ trong hợp đồng sự kiện.

Outbox và idempotency. Core ghi sự kiện vào `outbox_events` thay vì gửi thẳng; bộ phát chạy mỗi 2 giây đẩy tối đa 50 sự kiện, thử lại tối đa 5 lần rồi đánh dấu lỗi, nên lệnh ghi nghiệp vụ không phụ thuộc RabbitMQ có đang sống hay không. Thiết kế yêu cầu sự kiện được ghi cùng giao dịch với thay đổi nghiệp vụ; dịch vụ outbox đã nhận được giao dịch, nhưng các luồng hiện tại còn ghi job và sự kiện tuần tự, đây là điểm đang hoàn thiện. Phía nhận ghi cặp (mã thông điệp, bộ xử lý) vào `processed_messages` có ràng buộc duy nhất, trùng thì bỏ qua. Mỗi job có khoá idempotency duy nhất, ví dụ `{meetingId}:analyze:{transcriptVersionId}`, nên một phiên bản transcript chỉ được phân tích một lần.

2.6. Luồng xử lý từ âm thanh đến công việc

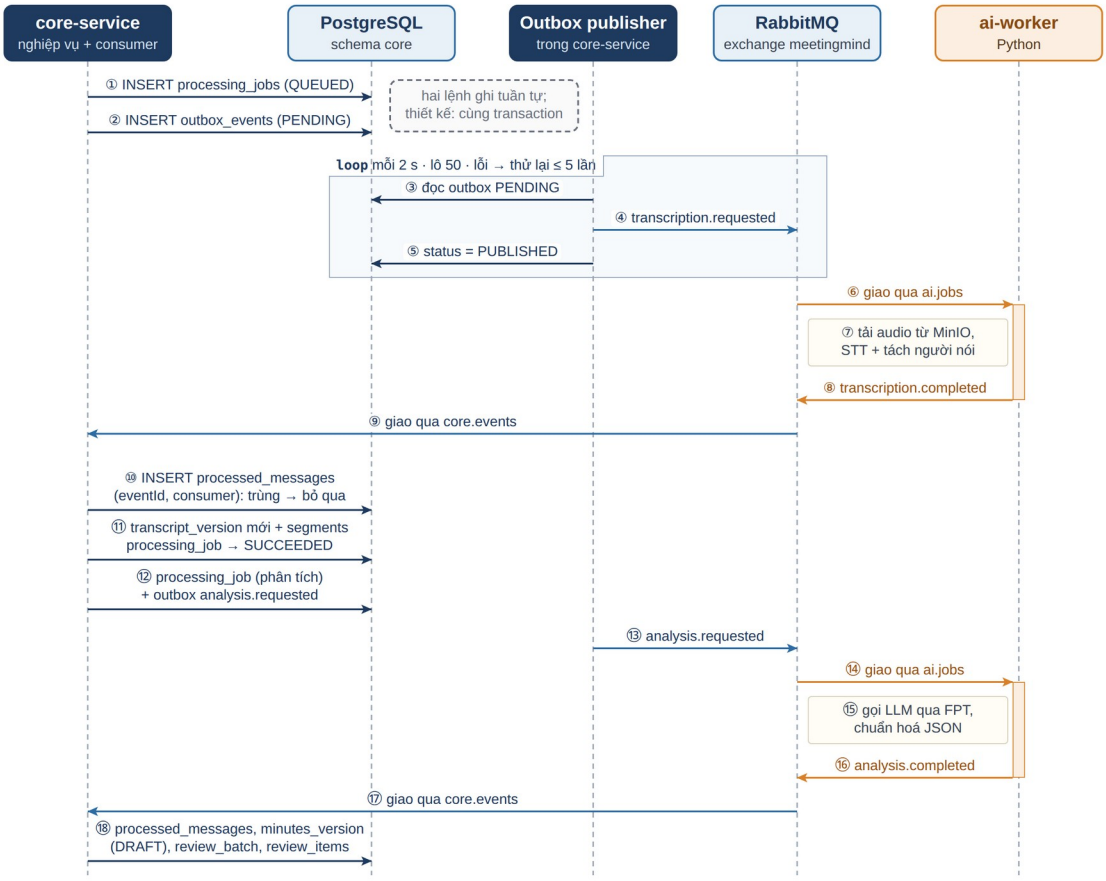
2.6.1. Luồng tổng quát

Hình 2.3 mô tả luồng xử lý, trong đó ba đường thu nhận hội tụ tại `transcript_versions`. Đường tải tệp và đường bot tạo tệp âm thanh trên MinIO: Core tạo job phiên âm, phát `transcription.requested`; ai-worker phiên âm bằng faster-whisper [19], tách người nói bằng vector giọng ERes2Net qua sherpa-onnx [4] [12], rồi phát `transcription.completed`. Đường phụ đề không có âm thanh: Core ghi từng lô phụ đề thành đoạn của một phiên bản nguồn `live_caption` và chốt khi người dùng kết thúc.



Hình 2.3. *Luồng xử lý từ âm thanh đến công việc*

Từ phiên bản transcript trở đi, ba đường dùng chung một luồng: Core phát `analysis.requested`; ai-worker gọi mô hình ngôn ngữ, phát `analysis.completed`; Core tạo biên bản nháp và một lô đề xuất gồm công việc, quyết định, vấn đề còn mở. Song song, mỗi nội dung chính thức (phiên bản transcript, biên bản đã duyệt, quyết định, công việc, tài liệu đính kèm cuộc họp) được Core gửi sang ai-worker để lập chỉ mục cho hỏi đáp. Chỉ khi được duyệt, đề xuất mới thành bản ghi chính thức và mới được integration-service đẩy sang Trello, báo qua Telegram; đồng bộ Jira vẫn là thiết kế. Hình 2.4 chi tiết hoá trình tự sự kiện qua outbox và bước chống lặp.



Hình 2.4. *Sơ đồ tuần tự sự kiện giữa core-service và ai-worker*

2.6.2. Hợp đồng sự kiện

Mọi sự kiện có phần bao chung gồm `eventId` (để chống lặp), `eventType`, `eventVersion`, `occurredAt`, `workspaceId` và `correlationId` (xuyên suốt một cuộc họp để lần vết), đi qua exchange dạng topic `meetingmind`. Bot-worker có hàng đợi riêng cho lệnh dừng, vì phiên ghi âm kéo dài hàng giờ. Bảng 2.8 liệt kê các sự kiện; mười sự kiện phiên âm, phân tích, đồng bộ công việc và lập chỉ mục tri thức được mô tả bằng JSON Schema dùng chung, bốn sự kiện của bot được định nghĩa đồng bộ trong mã TypeScript và Python. Cùng tệp hợp đồng còn đặc tả lời gọi HTTP nội bộ `POST /internal/knowledge/ask` từ Core sang ai-service cho hỏi đáp, vì người dùng chờ câu trả lời trực tiếp nên không đi qua hàng đợi.

Bảng 2.8. *Hợp đồng sự kiện giữa các dịch vụ*

Sự kiện	Bên phát → bên nhận	Nội dung chính
`transcription.requested`	Core → ai-worker	jobId, fileId, bucket, objectKey, language
`transcription.completed`	ai-worker → Core	runId, model, segments (nhãn, mốc thời gian, lời, cờ nói chồng)
`analysis.requested`	Core → ai-worker	jobId, transcriptVersionId, segments (số thứ tự, người nói, lời)
`analysis.completed`	ai-worker → Core	runId, model, summary, decisions,

Sự kiện	Bên phát → bên nhận	Nội dung chính
		openIssues, actionItems
`transcription.failed`, `analysis.failed`	ai-worker → Core	jobId, meetingId, error
`bot.join.requested`, `bot.stop.requested`	Core → bot-worker	botSessionId; meetingId, joinUrl (lệnh vào phòng)
`bot.status.changed`	bot-worker → Core	botSessionId, status, detail, error
`recording.completed`	bot-worker → Core	botSessionId, bucket, objectKey, durationMs, endReason
`task.sync.requested`	Core → integration-service	actionItemId, meetingId, idempotencyKey, task (tiêu đề, mô tả, người thực hiện, hạn, câu hạn gốc, ưu tiên, tên cuộc họp, câu trích nguồn)
`task.sync.completed`	integration-service → Core	actionItemId, results (provider, status, externalId, externalUrl, error)
`knowledge.index.requested`	Core → ai-worker	meetingId, meetingTitle, sourceType, sourceId, sourceVersion; items (nội dung có sẵn) hoặc document (bucket, objectKey, tên tệp)
`knowledge.delete.requested`	Core → ai-worker	meetingId, sourceType, sourceId (xoá các đoạn khớp)

Trường `assigneeHint` là *tên như xuất hiện trong transcript*, không phải mã tài khoản; Core không tự ánh xạ, để hai người trùng tên không bị gán nhầm tài khoản.

2.6.3. Trạng thái xử lý

Bảng `processing_jobs` là nguồn sự thật cho từng job, còn `processingStatus` của cuộc họp là giá trị tổng hợp để hiển thị nhanh; cuộc họp và phiên bot có vòng đời riêng, tóm tắt ở Bảng 2.9.

Bảng 2.9. *Trạng thái của job, cuộc họp và phiên bot*

Đối tượng	Chuyển trạng thái đang dùng
Job xử lý	queued → succeeded; queued → failed khi nhận sự kiện lỗi
Xử lý cuộc họp	transcribing → summarizing → done; failed khi một job lỗi
Diễn ra cuộc họp	scheduled → in_progress → ended; tải tệp thì tạo thẳng ở ended
Phiên bot	queued → joining → waiting_admission → recording → left; failed; cancelled

Cuộc họp chỉ chuyển "hoàn tất xử lý" khi biên bản nháp và hàng đợi duyệt đã được tạo. Sự kiện trạng thái bot tới trễ bị bỏ qua nếu phiên đã kết thúc. Trạng thái `running` của job và `indexing` của cuộc họp đã khai báo nhưng chưa dùng, vì ai-

worker chưa báo tiến độ trung gian và việc lập chỉ mục tri thức chạy độc lập, không gắn với trạng thái xử lý của cuộc họp.

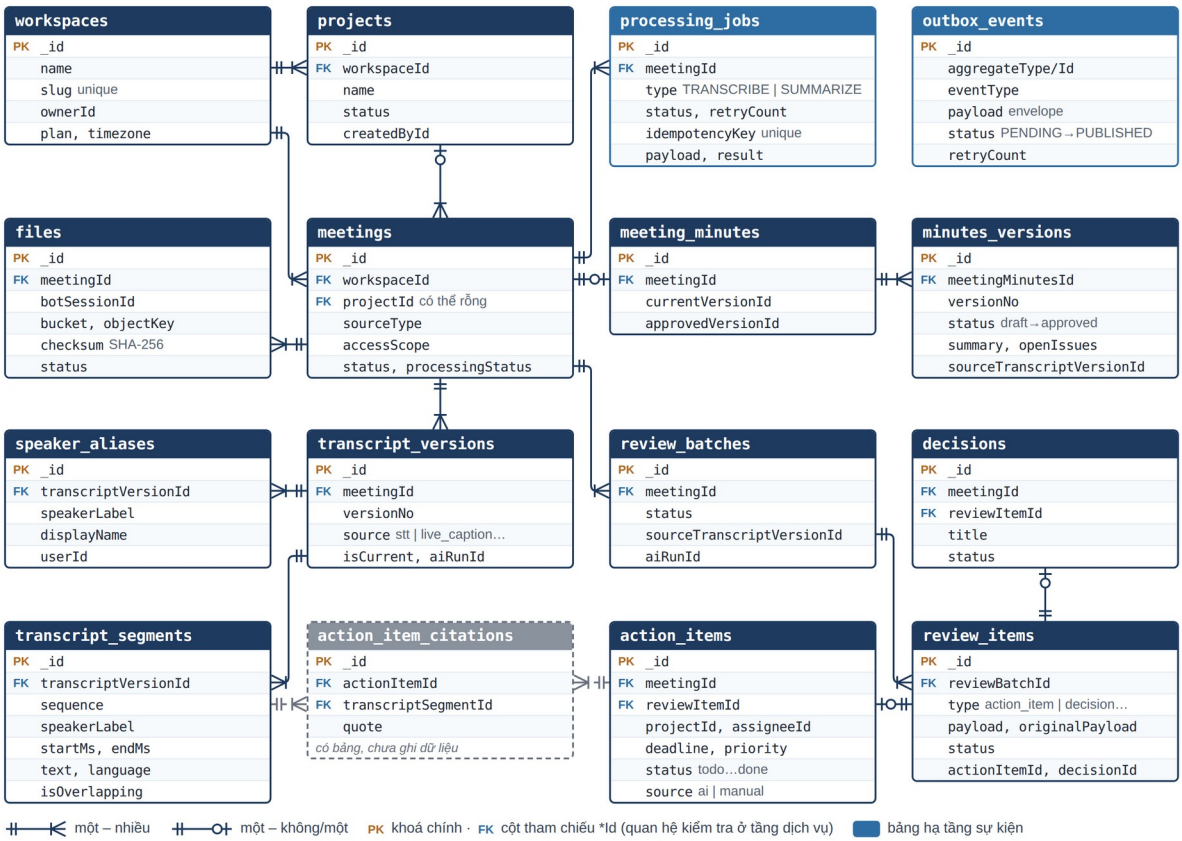
2.7. Thiết kế cơ sở dữ liệu

2.7.1. Tổ chức schema và tài khoản

Cơ sở dữ liệu dùng một máy chủ PostgreSQL có pgvector, chia ba schema ứng với ba tài khoản: `core` (`mm_core`), `integration` (`mm_integration`) và `ai` (`mm_ai`); mỗi tài khoản chỉ ghi được vào schema của mình. Tài liệu thiết kế gồm 43 bảng: 35 bảng `core`, 4 bảng `integration` và 4 bảng `ai` (lần chạy AI, đoạn tài liệu có vector nhúng, hội thoại hỏi đáp); khi chi tiết hoá phân quyền, chúng tôi bổ sung bảng `meeting_access`. Schema `core` đã hiện thực đầy đủ; schema `integration` đã tạo đủ bốn bảng cùng mô-đun đồng bộ; schema `ai` đã có bảng `document_chunks` lưu đoạn tri thức kèm vector nhúng 1024 chiều (pgvector 0.8.6, chỉ mục HNSW theo khoảng cách cosine) phục vụ hỏi đáp, còn bảng `ai_runs` ghi chi tiết từng lần chạy AI chưa được hiện thực. Hai bảng hội thoại `chat_threads` và `chat_messages` lưu lịch sử hỏi đáp theo từng người dùng. Khi hiện thực `document_chunks`, nhóm không sao chép quyền vào các cột `visibility`, `allowedUserIds` như thiết kế, mà để Core gửi danh sách cuộc họp người hỏi được xem kèm mỗi câu hỏi: quyền dùng để lọc luôn là quyền hiện hành, chỉ mục không phải cập nhật khi quyền thay đổi, đổi lại Core phải tính lại tập cuộc họp ở mỗi câu hỏi. Mọi bảng nghiệp vụ có `workspaceId` để tách dữ liệu theo workspace.

2.7.2. Các bảng chính

Hình 2.5 trình bày quan hệ các bảng chính của schema `core`, Bảng 2.10 mô tả các cột quan trọng. Cuộc họp là thực thể trung tâm: có nhiều phiên bản transcript và biên bản; mỗi lần phân tích tạo một lô duyệt; mỗi đề xuất đã duyệt sinh tối đa một công việc.



Hình 2.5. *Sơ đồ quan hệ các bảng chính của schema core*

Bảng 2.10. *Các bảng chính của schema core*

Bảng	Cột quan trọng
`meetings`	projectId, sourceType, accessScope, status, processingStatus, createdById
`transcript_versions`	versionNo, source, aiRunId, isCurrent; duy nhất (meetingId, versionNo)
`transcript_segments`	speakerLabel, speakerUserId, identityStatus, startMs, endMs, text, rawText, isOverlapping, sequence
`speaker_aliases`	speakerLabel, displayName, userId, confirmedById; duy nhất (transcriptVersionId, speakerLabel)
`meeting_minutes`, `minutes_versions`	currentVersionId, approvedVersionId; summary, openIssues, status, sourceTranscriptVersionId
`review_batches`, `review_items`	sourceTranscriptVersionId; type, originalPayload, payload, status, actionItemId
`action_items`	reviewItemId (duy nhất), assigneeId, deadline, deadlineRawText, deadlineNeedsConfirm, status, needsReviewReason
`action_item_citations`	transcriptSegmentId, transcriptVersionId, startMs, endMs, quote
`processing_jobs`	type, status, idempotencyKey (duy nhất), inputVersion, result
`outbox_events`, `processed_messages`	eventType, payload, status, retryCount; (messageId, consumer) duy nhất

2.7.3. Lưu phiên bản transcript và biên bản

Quyết định quan trọng nhất là lưu transcript và biên bản theo phiên bản, không ghi đè: nếu transcript bị sửa hoặc phân tích chạy lại, câu nguồn của công việc đã duyệt không được đổi nghĩa âm thầm. Mỗi lần nhận kết quả phiên âm, Core tạo phiên bản mới với số thứ tự tăng dần, ghi nguồn (`stt`, `live_caption`) và chuyển cờ `isCurrent` để đúng một phiên bản là hiện hành; mỗi đoạn lưu cả `text` hiện tại và `rawText` gốc. Biên bản đã duyệt không sửa trực tiếp mà phải mở phiên bản nháp mới rồi duyệt lại; chạy lại phân tích cũng chỉ sinh bản nháp mới. Tương tự, `review_items` lưu `originalPayload` (bản AI sinh) bên cạnh `payload` (sau khi sửa), vừa để truy vết vừa để đo mức người duyệt phải chỉnh sửa đầu ra của mô hình.

2.7.4. Nhãn người nói và bí danh

Mỗi đoạn giữ `identityStatus` với ba giá trị `unknown`, `suggested`, `confirmed`, bảo đảm nguyên tắc "không đủ căn cứ thì giữ chưa xác định". Bảng `speaker_aliases` ánh xạ nhãn sang tên hiển thị và tài khoản, kèm người và thời điểm xác nhận. Bí danh gắn với *phiên bản transcript* chứ không gắn với cuộc họp, vì tách người nói chạy lại có thể đánh số khác. Khi gửi transcript đi phân tích, Core thay nhãn đã có tên bằng tên đó. Khi gán lại người nói, hệ thống không sửa người thực hiện của công việc đã duyệt mà chỉ gắn lý do `speaker_changed` để rà soát.

2.7.5. Trích dẫn nguồn của công việc

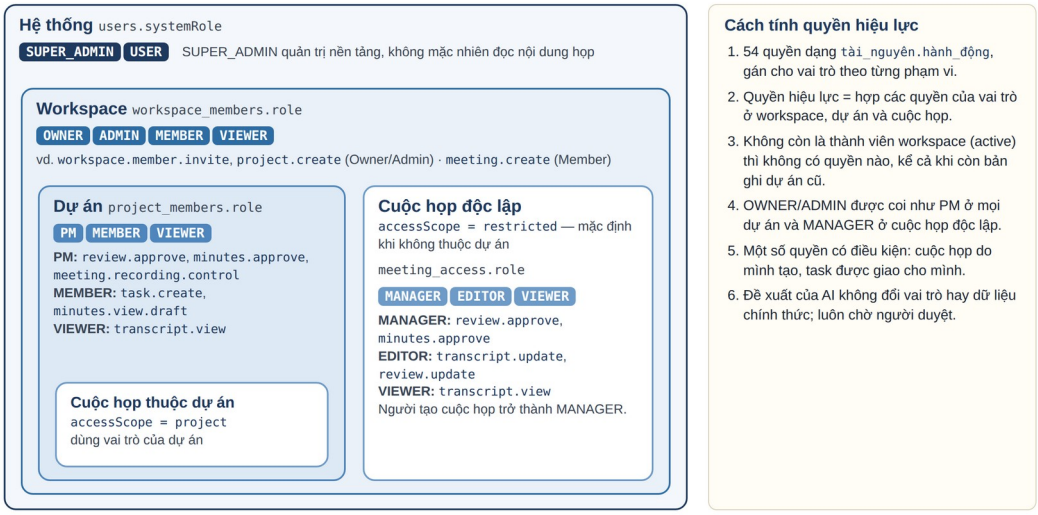
Bảng `action_item_citations` được thiết kế để lưu đoạn nguồn, *phiên bản transcript đã dùng lúc duyệt*, mốc thời gian và câu trích, để trích dẫn vẫn đúng khi transcript có phiên bản mới. Hiện tại chuỗi truy vết đi qua đề xuất: công việc có `reviewItemId`, đề xuất thuộc lô duyệt có `sourceTranscriptVersionId`, và nội dung đề xuất chứa câu trích cùng `transcriptSequence`, đủ xác định duy nhất đoạn nguồn. Việc tự sinh bản ghi trích dẫn khi duyệt là phần đang hoàn thiện.

2.8. Thiết kế phân quyền

2.8.1. Vai trò theo phạm vi

MeetingMind dùng phân quyền theo vai trò (RBAC) [21] kết hợp phạm vi tài nguyên, bổ sung điều kiện theo thuộc tính bản ghi theo tinh thần ABAC [10]. Hình 2.6 minh hoạ bốn tầng: hệ thống (`SUPER_ADMIN`, `USER`), workspace (`OWNER`, `ADMIN`, `MEMBER`, `VIEWER`), dự án (`PM`, `MEMBER`, `VIEWER`) và cuộc họp độc lập (`MANAGER`, `EDITOR`,

VIEWER). Cuộc họp thuộc dự án kế thừa quyền của dự án; cuộc họp độc lập mặc định ở phạm vi restricted và người tạo được gán MANAGER.



Hình 2.6. *Phân quyền theo vai trò kết hợp phạm vi tài nguyên*

Quyền được định nghĩa trong mã dạng <tài nguyên>.<hành động>, gồm 54 quyền; mỗi vai trò ánh xạ tới một tập quyền. Theo chính sách giai đoạn đầu, Owner/Admin workspace được coi như PM ở mọi dự án và MANAGER ở mọi cuộc họp độc lập. Ngược lại, SUPER_ADMIN không được đi vòng qua bộ kiểm tra quyền, vì quản trị nền tảng không đồng nghĩa với đọc nội dung khách hàng. Một số quyền không bao giờ được cấp qua vai trò, kể cả Owner: đọc hội thoại hỏi đáp hay lịch cá nhân của người khác, đọc khoá bí mật của kết nối tích hợp. Bảng 2.11 nêu các quyền tiêu biểu.

Bảng 2.11. *Một số quyền tiêu biểu và vai trò được phép*

Quyền	Vai trò được phép
workspace.member.invite, project.create, integration.manage	OWNER, ADMIN
meeting.create	MEMBER workspace trở lên
meeting.recording.control	PM, MANAGER; người tạo cuộc họp
transcript.update, review.update	PM, MANAGER, EDITOR; người tạo cuộc họp
review.approve, minutes.approve, task.assign	PM, MANAGER
task.status.update	PM, MANAGER; người được giao
sync.request	PM
chat.query	Mọi vai trò workspace

Các ô "người tạo cuộc họp" và "người được giao" là quyền có điều kiện, chỉ kiểm tra được khi đã biết bản ghi. Điều kiện "cuộc họp mình tạo" chỉ còn hiệu lực khi người đó vẫn xem được cuộc họp. Danh sách người tham dự chỉ ghi nhận ai có mặt, không cấp quyền truy cập.

2.8.2. Trình tự kiểm tra quyền

Việc kiểm tra tập trung trong `PermissionService` của Core, kích hoạt bởi `PermissionGuard` theo quyền khai báo trên từng route và luôn chạy sau bộ xác thực JWT. Trình tự gồm:

1. Xác thực token và nạp tài khoản.
2. Xác định phạm vi (workspace, dự án, cuộc họp) từ header, thân hoặc tham số request; request tải tệp phải truyền qua header vì thân chưa được đọc lúc bộ bảo vệ chạy.
3. Kiểm tra tư cách thành viên workspace ở trạng thái `active`; không có thì không có quyền nào, kể cả khi bản ghi thành viên dự án cũ còn.
4. Kiểm tra cuộc họp thuộc đúng workspace đang xét.
5. Tra vai trò dự án và vai trò cuộc họp độc lập, hợp các tập quyền, so với quyền yêu cầu.
6. Trong tầng dịch vụ, kiểm tra điều kiện theo bản ghi: người tạo, người được giao, chủ ghi chú.

So với thiết kế ban đầu, bước kiểm tra tài khoản hoặc workspace bị khoá chưa thành một bước riêng. Các danh sách "của tôi" cũng được lọc theo cùng tập quyền; hỏi đáp tri thức dùng lại đúng hàm lọc danh sách cuộc họp này để xác định những cuộc họp được phép tìm.

2.9. Thiết kế bộ phân tích AI

2.9.1. Quy tắc chống bịa trong prompt

Bộ phân tích dùng mô hình ngôn ngữ lớn [3] qua FPT AI Factory [7] với giao diện tương thích OpenAI; mô hình chính là DeepSeek-V4.1-Flash [5], dự phòng là GLM-5.2 [24]. Mô hình chính lỗi thì chuyển sang dự phòng; cả hai lỗi thì phát `analysis.failed`. Transcript đưa vào dạng các dòng đánh số `[số thứ tự] Người nói: lời nói`, để mô hình chỉ ra bằng chứng bằng một con số kiểm tra được. Lời nhắc hệ thống đặt mô hình vào vai thư ký và nêu các quy tắc chống bịa [11]:

- Quyết định chỉ gồm điều cả nhóm đã chốt; đề xuất chưa ai đồng ý ("hay là…") không phải quyết định.
- Người thực hiện ghi đúng tên trong transcript; không rõ thì để trống.
- Hạn chót ghi nguyên văn ("thứ Sáu", "18/11"); không có thì để trống, không tự đoán ngày.
- Có đính chính (đổi hạn, đổi người, huỷ việc) thì lấy thông tin sau cùng; bỏ qua chuyện ngoài lề.

– Mỗi mục kèm câu trích nguyên văn trong đúng dòng được nêu; không có gì thì trả mảng rỗng.

2.9.2. Cấu trúc đầu ra

Mô hình chỉ trả một đối tượng JSON như Bảng 2.12. Chúng tôi không bật chế độ ép JSON của API, vì khi bật, DeepSeek-V4.1-Flash trả nội dung rỗng; bộ phân tích tự bóc đối tượng JSON từ câu trả lời và tự kiểm tra cấu trúc.

Bảng 2.12. *Cấu trúc JSON đầu ra của bộ phân tích*

Trường	Ý nghĩa
`summary`	Tóm tắt 3–5 câu
`decisions[]`	title, description, quote, transcriptSequence
`openIssues[]`	Vấn đề chưa có kết luận
`actionItems[].assigneeHint`	Tên người thực hiện như trong transcript, hoặc rỗng
`actionItems[].deadlineRawText`	Cụm chỉ thời hạn nguyên văn, hoặc rỗng
`actionItems[].priority`, `confidence`	high/medium/low; số trong [0, 1]
`actionItems[].quote`, `transcriptSequence`	Câu trích làm bằng chứng và số thứ tự dòng

Kết quả sau đó được chuẩn hoá: ưu tiên không hợp lệ đưa về `medium`; độ tin cậy ép vào [0, 1]; các chuỗi "null", "N/A" đưa về rỗng thật; mục thiếu tiêu đề bị loại. Quan trọng nhất là bước *đối chiếu bằng chứng*: mô hình đôi khi trích đúng câu nhưng ghi lệch số dòng, nên bộ phân tích so câu trích với dòng được nêu, không khớp thì dò lại toàn transcript, vẫn không thấy thì chỉ giữ số dòng nếu số đó tồn tại. Transcript rỗng thì không gọi mô hình.

2.9.3. Con người trong vòng lặp

Đầu ra không bao giờ thành dữ liệu chính thức ngay: Core chuyển nó thành biên bản nháp và các đề xuất giữ bản gốc; người duyệt xem đề xuất cạnh câu nguồn, chọn người thực hiện từ danh sách thành viên (`assigneeHint` chỉ để tham khảo), chọn ngày hạn rồi duyệt hoặc từ chối. Mô hình được coi như người soạn nháp nhanh nhưng có thể sai, vì một công việc giao nhầm người hay nhầm hạn gây hậu quả lớn hơn nhiều so với vài giây kiểm tra.

2.9.4. Yêu cầu thiết kế cho hỏi đáp tri thức

Tài liệu tổng quan dự án đặt cho chức năng hỏi đáp trên kho cuộc họp các quy tắc sau, theo cùng tinh thần chống bịa của bộ phân tích. Câu trả lời phải kèm nguồn: cuộc họp, loại nguồn và thời điểm trong transcript; không đủ nguồn thì trả lời "chưa đủ thông tin" thay vì suy đoán. Quyền phải được lọc *trước* khi truy xuất: lọc theo

workspace là chưa đủ, vì thành viên workspace vẫn có thể không được xem một dự án hay một cuộc họp riêng tư, nên chỉ nội dung của những cuộc họp người hỏi được xem mới được đưa vào ngữ cảnh của mô hình; quyền còn được kiểm tra lại khi trả nguồn về. Khi nguồn được sửa, có phiên bản mới hoặc bị xoá, chỉ mục phải được cập nhật theo. Về phân công, AI không biết gì về quyền: Core quyết định người hỏi được tìm trong những cuộc họp nào, dịch vụ AI chỉ tìm và trả lời trong phạm vi được giao. Thiết kế dữ liệu và quyền này được hiện thực như trình bày ở chương Xây dựng hệ thống.

2.10. Thiết kế giao diện

Giao diện web xây dựng bằng Next.js với 29 trang, nhóm như Bảng 2.13. Giao diện có chế độ nối backend thật và chế độ dữ liệu mẫu để trình diễn các chức năng chưa có backend; cột cuối của bảng ghi rõ mức hoàn thiện.

Bảng 2.13. *Các màn hình chính của giao diện web*

Nhóm	Trang	Mức hoàn thiện
Xác thực	Đăng nhập, Đăng ký	Nối API
Tổng quan	Dashboard	Phần lớn nối API
Cuộc họp	Tất cả, Sắp tới, Chờ duyệt, Đang xử lý, Hoàn tất, Ghi âm cá nhân, Chi tiết cuộc họp	Nối API
	Ghi âm trực tiếp	Giao diện mô phỏng
Dự án	Danh sách, Chi tiết	Một phần
Công việc	Task cần duyệt, Task board, Task tồn, Cảnh báo	Nối API
Tri thức	Tri thức: notebook nguồn đã lập chỉ mục, hỏi đáp có trích dẫn, tải tài liệu	Nối API
Khách mời	Cuộc họp được mời	Giao diện mẫu
Cài đặt	Tài khoản, Thành viên và quyền	Nối API
	Mẫu giọng, Tích hợp, Mẫu biên bản, Lưu trữ, Nhật ký	Giao diện mẫu
Điều hướng	Trang gốc, /work, /settings	Chuyển hướng

Thanh điều hướng ẩn mục theo quyền chỉ để tiện dụng; kiểm soát thật vẫn ở backend. Luồng điển hình của PM: tạo cuộc họp qua hộp thoại; tải tệp thì phiên âm khởi động ngay, họp trực tuyến thì bấm "Mời bot" và theo dõi trạng thái bot; xử lý xong thì mở tab transcript, nơi đoạn nói chồng có nhãn "nên nghe lại", để gán tên người nói; tiếp theo duyệt biên bản và đề xuất; cuối cùng theo dõi công việc trên Task board Kanban bốn cột, mỗi thẻ mở được câu nguồn và bình luận. Thành viên chỉ thấy "Việc của tôi" để cập nhật tiến độ. Với tiện ích Chrome, người dùng bấm một nút

trong phòng Google Meet; tiện ích tự tìm hoặc tạo cuộc họp theo đường dẫn phòng, gửi phụ đề theo lô 5 giây và chốt transcript khi kết thúc.

2.11. Thiết kế module đồng bộ công cụ ngoài và cảnh báo

2.11.1. Hiện trạng và dữ liệu

Module đồng bộ công việc sang Jira [1] hoặc Trello [2] và gửi cảnh báo qua Telegram [25] thuộc integration-service. Đến thời điểm nghiệm thu, phần Trello và Telegram đã được hiện thực ở mức MVP (trình bày ở chương Xây dựng hệ thống): integration-service nhận `task.sync.requested`, tạo thẻ Trello, gửi tin Telegram rồi phát `task.sync.completed`; bốn bảng của schema `integration` ở Bảng 2.14 đã được tạo. Phần còn ở mức thiết kế gồm đồng bộ Jira, màn hình kết nối theo từng workspace (trang "Tích hợp" vẫn là giao diện mẫu) và đồng bộ hai chiều trạng thái. Mọi trường trỏ về Core đều là tham chiếu logic, không có khoá ngoại xuyên schema.

Bảng 2.14. *Các bảng của schema integration*

Bảng	Cột quan trọng	Vai trò
`integrations`	provider, status, config, lastError	Một kết nối với một nhà cung cấp; `config` chỉ giữ thông tin không bí mật (danh sách Trello đã dò, cuộc trò chuyện Telegram)
`external_task_links`	integrationId, actionItemId, externalId, externalUrl, syncStatus	Liên kết công việc với thẻ ngoài; duy nhất (integrationId, actionItemId)
`sync_jobs`	type, status, idempotencyKey, payload, result, retryCount, error	Một lần đẩy công việc ra một nhà cung cấp; duy nhất idempotencyKey
`processed_messages`	messageId, consumer	Chống xử lý lặp thông điệp RabbitMQ; duy nhất (messageId, consumer)

Luồng thiết kế: khi công việc được duyệt, Core ghi `task.sync.requested` vào outbox; integration-service tạo job, gọi REST API của nền tảng đích, ghi liên kết ngoài rồi phát `task.sync.completed`. Khi hiện thực, nhóm điều chỉnh một điểm so với thiết kế ban đầu: thay vì để integration-service gọi ngược API nội bộ của Core để đọc công việc, thông điệp mang sẵn ảnh chụp công việc, tránh phụ thuộc đồng bộ giữa hai dịch vụ. Theo thiết kế đầy đủ, kết nối dùng OAuth 2.0 [9] nơi nhà cung cấp hỗ trợ, token được mã hoá khi lưu và không bao giờ được trả về qua API hay giao diện; bản MVP hiện tại lấy khoá API từ biến môi trường và không ghi khoá vào cơ sở dữ liệu.

2.11.2. Nguyên tắc chống tạo trùng và cảnh báo

Hai nguyên tắc chi phối đồng bộ. Thứ nhất, *chỉ đồng bộ công việc đã duyệt*, vì một thẻ hay issue sai trên công cụ ngoài khó thu hồi hơn một đề xuất bị từ chối. Thứ

hai, *thử lại không tạo bản trùng*: chống lặp thông điệp nội bộ là chưa đủ, vì lời gọi tạo issue có thể đã thành công nhưng phản hồi bị mất, gửi lại sẽ tạo issue thứ hai. Với API không hỗ trợ khoá idempotency, thiết kế yêu cầu tra cứu đối soát trước khi thử lại; ràng buộc duy nhất trên (kết nối, công việc) là chốt chặn cuối. Bản hiện thực cho Trello áp dụng đúng nguyên tắc này bằng một dòng đánh dấu trong mô tả thẻ; phạm vi đã hiện thực là đồng bộ một chiều (chỉ tạo thẻ).

Cảnh báo dựa trên quy tắc minh bạch thay vì phán đoán của AI. Bảng `risk_flags` đã có trong schema `core`, lưu quy tắc sinh cảnh báo (`ruleKey`), dữ liệu căn cứ (`evidence`) và khoá chống tạo trùng qua các lần quét. Hiện giao diện tính năm loại cảnh báo trên dữ liệu công việc (quá hạn, bị chặn, thiếu người phụ trách, phụ thuộc chưa xong, từ năm hạn dồn vào một ngày cho một người), mỗi cảnh báo kèm quy tắc tương ứng; chỉ kết luận "quá tải" khi có dữ liệu công sức và năng lực. Kênh Telegram hiện mới dùng để báo công việc mới được duyệt; bộ quét phía máy chủ và các cảnh báo, nhắc việc theo quy tắc (quá hạn, sắp đến hạn) qua Telegram chưa được hiện thực.

2.12. Kết luận chương

Chương này đã phân tích yêu cầu và trình bày thiết kế của MeetingMind. Từ khảo sát, chúng tôi rút ra quy tắc cốt lõi: phân biệt đề xuất với cam kết, không bịa người hay hạn, mọi công việc có nguồn, chỉ công việc đã duyệt mới được giao. Về kiến trúc, Core là bên điều phối và là nơi duy nhất quyết định quyền; bot và AI chỉ thực thi và báo kết quả qua RabbitMQ, với outbox, chống xử lý lặp và khoá idempotency. Cơ sở dữ liệu lưu transcript và biên bản theo phiên bản để công việc luôn truy được về đúng câu nguồn; bộ phân tích AI bị ràng buộc bằng quy tắc chống bịa và luôn qua bước duyệt của con người. Phần lõi từ thu nhận tới bảng công việc đã hiện thực, cùng đồng bộ một chiều sang Trello và thông báo công việc mới qua Telegram; hỏi đáp có trích nguồn trên tri thức cuộc họp với lọc quyền trước truy xuất cũng đã được hiện thực; đồng bộ Jira, cảnh báo theo quy tắc phía máy chủ và mã hoá khi lưu còn ở mức thiết kế. Chương tiếp theo trình bày việc hiện thực các thiết kế này.

CHƯƠNG 3. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ

3.1. Đối chiếu với kết quả cần đạt

Bảng 3.1. *Đối chiếu kết quả Nội dung 2 với đề cương*

Kết quả cần đạt theo đề cương	Kết quả thực hiện	Đánh giá
Tài liệu đặc tả yêu cầu (SRS)	Tài liệu tổng quan dự án 15 mục: tác nhân, yêu cầu chức năng và phi chức năng, quy trình họp mẫu, màn hình, tiêu chí nghiệm thu, quy tắc chống AI bịa việc	Đạt
Sơ đồ kiến trúc hệ thống	Kiến trúc 5 dịch vụ backend, frontend và extension; giao tiếp đồng bộ qua HTTP và bất đồng bộ qua RabbitMQ; nguyên tắc outbox, idempotency, AI chỉ đề xuất	Đạt
Thiết kế cơ sở dữ liệu	43 bảng trong 3 schema (core, integration, ai) với 3 tài khoản riêng; transcript và biên bản lưu theo phiên bản	Đạt (schema core và integration đã hiện thực; ai mới ở mức thiết kế)
Luồng dữ liệu từ Audio đến Task	Luồng 10 bước từ thu nhận đến công việc chính thức; hợp đồng sự kiện dùng chung giữa TypeScript và Python	Đạt
Thiết kế luồng xử lý prompt	Quy tắc chống bịa, cấu trúc JSON đầu ra, đối chiếu bằng chứng, con người duyệt	Đạt

Thiết kế ở bước này là nền tảng ổn định cho các bước sau: khi thay bộ phiên âm và bộ phân tích giả lập bằng Whisper và LLM thật ở cuối đề tài, luồng điều phối trong core-service gần như không phải sửa, đúng với mục tiêu tách biệt đã đặt ra. Thiết kế module đồng bộ cũng được dùng gần như nguyên vẹn khi hiện thực đồng bộ Trello và Telegram (xem báo cáo Nội dung 4), chỉ điều chỉnh để thông điệp mang sẵn ảnh chụp công việc thay vì gọi ngược API của Core. Một số điểm thiết kế chưa được hiện thực đầy đủ và được ghi nhận trung thực trong chương thiết kế: ghi outbox cùng giao dịch với dữ liệu nghiệp vụ, đường đi qua API Gateway, schema ai, phần Jira và kết nối OAuth theo workspace của module đồng bộ.

TÀI LIỆU THAM KHẢO

[1] Atlassian (2026), *Jira Cloud platform REST API v3*, https://developer.atlassian.com/cloud/jira/platform/rest/v3/, truy cập ngày 04/11/2026.

[2] Atlassian (2026), *Trello REST API*, https://developer.atlassian.com/cloud/trello/rest/, truy cập ngày 04/11/2026.

[3] Brown T. B., Mann B., Ryder N., Subbiah M., Kaplan J., et al. (2020), "Language models are few-shot learners", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 1877–1901.

[4] Chen Y., Zheng S., Wang H., Cheng L., Chen Q., Qi J. (2023), "An enhanced Res2Net with local and global feature fusion for speaker verification", *Proceedings of Interspeech 2023*.

[5] DeepSeek-AI (2024), *DeepSeek-V3 Technical Report*, arXiv:2412.19437.

[6] Fireflies.ai (2026), *Fireflies.ai – AI notetaker for meetings*, https://fireflies.ai, truy cập ngày 04/11/2026.

[7] FPT Smart Cloud (2026), *FPT AI Factory – Serverless Inference documentation*, https://docs.fptcloud.com/docs/category/fpt-ai-factory/, truy cập ngày 04/11/2026.

[8] Google (2026), *Chrome Extensions – Manifest V3*, https://developer.chrome.com/docs/extensions, truy cập ngày 04/11/2026.

[9] Hardt D. (Ed.) (2012), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF).

[10] Hu V. C., Ferraiolo D., Kuhn R., Schnitzer A., Sandlin K., Miller R., Scarfone K. (2014), *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162.

[11] Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. (2023), "Survey of hallucination in natural language generation", *ACM Computing Surveys*, 55(12), pp. 1–38.

[12] k2-fsa (2026), *sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization using next-gen Kaldi with onnxruntime*, https://github.com/k2-fsa/sherpa-onnx, truy cập ngày 04/11/2026.

[13] Kleppmann M. (2017), *Designing Data-Intensive Applications*, O'Reilly Media.

[14] Microsoft (2026), *Playwright documentation*, https://playwright.dev, truy cập ngày 04/11/2026.

[15] Myśliwiec K. (2026), *NestJS documentation*, https://docs.nestjs.com, truy cập ngày 04/11/2026.

[16] Newman S. (2021), *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly Media.

[17] Otter.ai (2026), *Otter.ai – AI meeting assistant*, https://otter.ai, truy cập ngày 04/11/2026.

[18] RabbitMQ (2026), *RabbitMQ documentation: Reliability guide, consumer acknowledgements and publisher confirms*, https://www.rabbitmq.com/docs, truy cập ngày 04/11/2026.

[19] Radford A., Kim J. W., Xu T., Brockman G., McLeavey C., Sutskever I. (2023), "Robust speech recognition via large-scale weak supervision", *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 28492–28518.

[20] Richardson C. (2018), *Microservices Patterns: With Examples in Java*, Manning Publications.

[21] Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. (1996), "Role-based access control models", *IEEE Computer*, 29(2), pp. 38–47.

[22] SYSTRAN (2026), *faster-whisper: Faster Whisper transcription with CTranslate2*, https://github.com/SYSTRAN/faster-whisper, truy cập ngày 04/11/2026.

[23] Tactiq (2026), *Tactiq – AI meeting transcripts for Google Meet, Zoom and Teams*, https://tactiq.io, truy cập ngày 04/11/2026.

[24] Team GLM (2024), *ChatGLM: A family of large language models from GLM-130B to GLM-4 All Tools*, arXiv:2406.12793.

[25] Telegram (2026), *Telegram Bot API*, https://core.telegram.org/bots/api, truy cập ngày 04/11/2026.